

Github Actions Comprehensive Guide



2025

[Click here for DevSecOps & Cloud DevOps Course](#)

DevOps Shack

GitHub Comprehensive Guide

Ultimate GitHub Actions CI/CD Pipeline Guide

GitHub Actions is GitHub's CI/CD platform that allows you to automate, customize, and execute your software development workflows right in your GitHub repository.

Why Use GitHub Actions?

 Feature	 Explanation
 Native to GitHub	No external CI/CD setup needed
 Automates Testing	Run unit, integration, and e2e tests
 Continuous Deployment	Deploy to cloud, Kubernetes, or servers
 Artifact Management	Share builds/logs between jobs
 Secure Secrets Handling	Encrypted secrets and environment variables
 Cross-platform	Supports Ubuntu, Windows, and macOS runners

Key Components

1. Workflow

A **workflow** defines the entire automation pipeline.

 Location: .github/workflows/workflow-name.yml

 Structure:

name: My CI Pipeline

on: push

jobs: ...

2. Events (Triggers)

These define **when** a workflow runs.

Trigger	Description
push	On code push
pull_request	When a PR is opened/synced
schedule	Cron-based jobs (e.g., nightly builds)
workflow_dispatch	Manual trigger from GitHub UI
release	On release creation

Example:

```
on:  
  push:  
    branches: [main]  
  pull_request:  
  schedule:  
    - cron: "0 0 * * *"
```

3. Jobs

Each **job** is a group of steps that run on a specific runner (machine).

Example:

```
jobs:  
  test:  
    runs-on: ubuntu-latest  
    steps:  
      - run: echo "Running tests"
```

4.Steps

Each **step** is an individual task in a job, like:

- Running shell scripts

- Using GitHub Actions
- Installing dependencies

 Example:

steps:

```
- name: Print Hello  
  run: echo "Hello, World!"
```

5. Runners

A **runner** is a server (hosted by GitHub or yourself) that executes your workflow.

Type	OS Options	Notes
ubuntu-latest	Linux	Most commonly used
windows-latest	Windows	Useful for .NET/Windows apps
macos-latest	macOS	Useful for iOS/macOS builds
self-hosted	Any	For custom hardware/software needs

6. Actions

An **action** is a pre-built, reusable module you can include in workflows.

 Categories:

- Official (maintained by GitHub)
- Community (published by developers)
- Custom (you can write your own)

 Example:

```
- uses: actions/checkout@v4  
- uses: actions/setup-node@v4  
  with:  
    node-version: '18'
```

7. Secrets & Variables

- **Secrets:** Encrypted values stored in repo settings
- **Variables:** Environment-specific values (region, env, etc.)

 Example:

env:

REGION: \${{ vars.AWS_REGION }}

- name: Login

run: echo "My secret is \${{ secrets.MY_SECRET }}"

Real-World Examples

Simple Node CI Workflow

name: Node CI

on: push

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with:

node-version: '18'

- run: npm install

- run: npm test

Docker Build & Push

jobs:

docker:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: docker/login-action@v3

with:

username: \${{ secrets.DOCKER_USER }}

password: \${{ secrets.DOCKER_PASS }}

- uses: docker/build-push-action@v6

```
with:  
  context: .  
  push: true  
  tags: user/app:latest
```

Deploy to Kubernetes

```
- name: Deploy to EKS  
  run: kubectl apply -f k8s/
```

Advanced Concepts

Matrix Strategy

Run jobs for multiple versions/platforms:

```
strategy:  
  matrix:  
    node: [14, 16, 18]  
  steps:  
    - uses: actions/setup-node@v4  
      with:  
        node-version: ${{ matrix.node }}
```

Job Dependencies

Control execution order using needs::

```
jobs:  
  build:  
    runs-on: ubuntu-latest  
  test:  
    runs-on: ubuntu-latest  
    needs: build
```

Reusable Workflows

Split common logic into **reusable templates**:

uses: `./.github/workflows/deploy-template.yml`

Environments with Protection Rules

Use GitHub **Environments** for staged deployments (dev, staging, prod).
Each environment can have:

- Review gates
 - Secret scopes
 - Approvals
-

Observability

-  **View logs** in real time via the Actions tab
 -  **Test summaries** and artifacts
 -  **Failure alerts** (add Slack or Discord integration)
-

Best Practices

- ✓ Use small, modular workflows
- ✓ Reuse workflows or composite actions
- ✓ Always pin action versions (@v4 not @latest)
- ✓ Secure secrets and limit their access
- ✓ Use caching to speed up builds
- ✓ Automate everything: test, build, deploy

Project Tutorial:

https://www.youtube.com/watch?v=icZUzgtz_d8&pp=0gcJCb8Ag7Wk3p_U

This guide walks through setting up a **multi-stage CI/CD pipeline** using **GitHub Actions**. It includes stages like:

-  **Compilation**
-  **Security Scanning** (Trivy + Gitleaks)
-  **Unit Testing**
-  **SonarQube Analysis**
-  **Docker Image Build & Push**
-  **Kubernetes Deployment (EKS)**

CICD Pipeline Trigger

on:

push:

branches: ["main"]

📌 **Explanation:** This pipeline triggers **every time code is pushed** to the main branch. You can customize this to support other events like PRs, tags, or schedule-based triggers.

✅ **Job 1: Compilation**

compile:

runs-on: self-hosted

Steps:

-  **Checkout Code:** Fetch your source code from GitHub.
-  **Setup Java 17** using temurin distribution.
-  **Compile the Project** with Maven.

- uses: actions/checkout@v4

- name: Set up JDK 17

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

cache: maven

- name: Build with Maven

run: mvn compile

✅ **Purpose:** Ensures that your code compiles successfully before moving forward.

Job 2: Security Checks (Trivy + Gitleaks)

security-check:

runs-on: self-hosted

needs: compile

Steps:

- ✅ **Install Trivy** (file system scanning for vulnerabilities).
- 🔍 **Scan the project folder** using Trivy.
- 🔑 **Install Gitleaks** (secrets scanning).
- 📄 **Scan for exposed secrets** in your repo.

- name: Trivy FS Scan

run: trivy fs --format table -o fs-report.json .

- name: Gitleaks Code Scan

run: gitleaks detect source . -r gitleaks-report.json -f json

📦 **Output:** Reports generated for vulnerabilities and secrets.

Job 3: Unit Testing

test:

runs-on: self-hosted

needs: security-check

Steps:

- Setup Java.
- Run mvn test to execute unit tests.

 **Why this matters:** Catch bugs early by ensuring business logic works as expected.

Job 4: Build & SonarQube Analysis

build_project_and_sonar_scan:

runs-on: self-hosted

needs: test

Steps:

1. **Build JAR** using mvn package.
2. **Upload Artifact** so it can be used by the Docker job.
3. **SonarQube Analysis:**
 - Scan code.
 - Wait for **quality gate** pass/fail status.

- name: Upload JAR artifact

uses: actions/upload-artifact@v4

with:

name: app-jar

path: target/*.jar

 **SonarQube** helps you maintain **code quality**, **coverage**, and **security standards**.

 Requires:

- SONAR_TOKEN (Secret)

-
- SONAR_HOST_URL (Variable)
-

Job 5: Build & Push Docker Image

build_docker_image_and_push:

runs-on: self-hosted

needs: build_project_and_sonar_scan

Steps:

- Download JAR from previous stage.
- Login to Docker Hub.
- Setup Docker Buildx.
- Build and push the Docker image.

- name: Build and Push Docker image

uses: docker/build-push-action@v6

with:

context: .

push: true

tags: adijaiswal/bankapp:latest

file: ./Dockerfile

 **Tagging** is crucial for versioning and rolling deployments.

Job 6: Deploy to Kubernetes (EKS)

deploy_to_kubernetes:

runs-on: self-hosted

needs: builds_docker_image_and_push

Steps:

- Install AWS CLI and kubectl.
- Configure AWS credentials.

- Setup kubeconfig using secret.
- Deploy using a Kubernetes manifest (ds.yml).

- name: Configure kubeconfig

run: |

mkdir -p \$HOME/.kube

echo "\${ secrets.EKS_KUBECONFIG }}" > \$HOME/.kube/config

📌 **Tip:** Make sure your EKS cluster and IAM roles are properly set.

🔧 Secret & Variable Management

Name	Type	Used In
SONAR_TOKEN	Secret	SonarQube
SONAR_HOST_URL	Variable	SonarQube
DOCKERHUB_USERNAME	Variable	Docker login
DOCKERHUB_TOKEN	Secret	Docker login
AWS_ACCESS_KEY_ID	Secret	AWS Auth
AWS_SECRET_ACCESS_KEY	Secret	AWS Auth
EKS_KUBECONFIG	Secret	EKS Deployment

🔄 Workflow Dependency Graph

```

graph TD
    compile --> security-check
    security-check --> test
    test --> build_project_and_sonar_scan
    build_project_and_sonar_scan --> builds_docker_image_and_push
    builds_docker_image_and_push --> deploy_to_kubernetes
  
```

✓ Best Practices

- ✦ **Caching Maven** dependencies speeds up builds.
- ✦ Use **Artifacts** to transfer files between jobs.
- ✦ Ensure **secrets are encrypted** and not logged.
- ✦ Enable **GitHub branch protection rules** to enforce passing checks.

📦 Bonus Tips

-  Add **Slack or Email notification** actions after deploy.
-  Include **integration or e2e tests** in a separate job.
-  Use **GitHub Environments** for staging and prod segregation.
-  Integrate **Ingress** for external access to your app.